> ACTIVIDAD 20 <

Análisis de resultados: Proyecto Final

García Guerra Juan Adrián ORCID: 0009-0006-0244-8871

INTRODUCCIÓN

Se analizarán cada una de las cajas que componen el código del proyecto final. Se explicará qué hace el código y cuál es su resultado. También se analizarán los resultados que se generan en cada caja en el contexto de los datos recolectados para los servicios Uber y Lyft.

.

Caja 0. Importación de librerías y asignación de ruta del dataset



Fig. 1. Librerías y asignación de la ruta del dataset

En esta primera caja estamos importando las librerías necesarias para la realización del análisis de datos. Las librerías numpy, pandas y matplotlib nos proporcionan soporte para matrices y operaciones matemáticas en arrays multidimensionales, manipulación y análisis de datos, y visualización de datos respectivamente. Mientras que os y ziplfile nos proporcionan funciones para interactuar con el sistema operativo y clases para leer y escribir archivos zip respectivamente. La librería sklearn nos proporciona herramientas eficientes para el análisis predictivo y el aprendizaje automático. Aquí estamos importando funciones que nos permitirán ajustar modelos, en este caso una regresión logística, entrenar los datos, escalar, obtener estadísticas descriptivas, entre otras que nos ayudarán en nuestro proyecto.

En la última línea de código, estamos asignando el valor "datasets" a la variable **PATH**. Este valor indica la ruta o la ubicación donde se encuentran los conjuntos de datos que se utilizarán en el código posteriormente, y en donde **PATH** se convierte en una referencia conveniente de acceder a la carpeta que contiene los conjuntos de datos

Caja 1. Preparación de los datos y carga del dataset

En esta caja empezamos por abrir el archivo zip en modo lectura ubicado en la ruta que especificamos anteriormente. Todo esto dentro del bloque with, en el cuál también extraemos todos los archivos y directorios que contenga el archivo zip. Después de extraer los archivos del archivo zip, se utiliza la función **listdir()** del módulo **os** para listar el contenido del directorio especificado por **PATH**. Esto devuelve una lista de los nombres de los archivos y directorios en el directorio especificado.



Fig. 2. Preparación de los datos y carga del dataset



Fig. 3. Nombres de los archivos contenidos en el archivo zip

Después las siguientes líneas de código nos permiten cargar nuestro dataset en un dataframe llamado **Ask**. Luego con la llamada a la función **Ask.head()** podremos ver las primeras filas del dataframe que hemos creado.

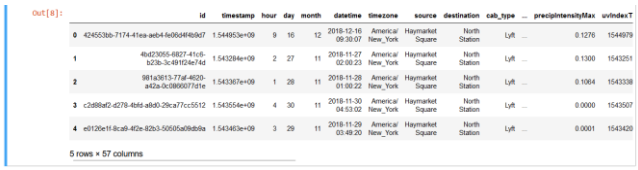


Fig. 4. Primeras filas de nuestro dataset cargado

Caja 2. Análisis exploratorio de datos (EDA)

En esta caja tendremos por objetivo analizar nuestros datos para realizar los ajustes necesarios antes de pasar a la visualización. Así, llevaremos a cabo procesos de limpieza, transformación y depuración de los datos. Entonces comenzamos por llamar a la función **Ask.info()** para obtener un resumen detallado del dataframe creado.

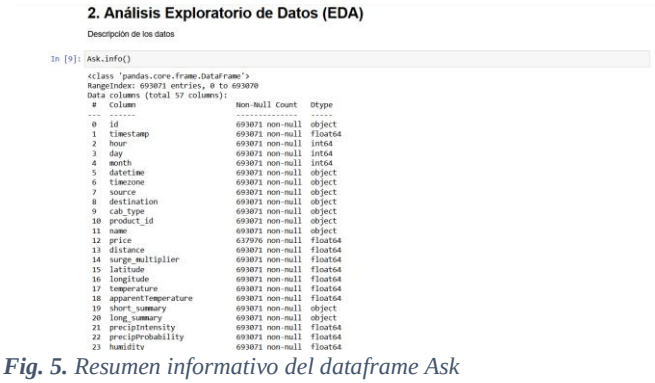


Fig. 5. Resumen informativo del dataframe Ask

> ACTIVIDAD 20 <

A continuación se realizan los procesos de llenado de datos nulos y la depuración de las columnas que no necesitemos. Para el llenado de datos nulos, estos se rellenaron con la media de los valores existentes de la columna ‘price’, que es donde requerimos ese relleno. Después creamos la variable **columns_to_drop** para llevar a cabo el proceso de eliminación de columnas, las cuales especificamos en la variable antes creada. Por último llamamos a la función **Ask.describe()** para obtener estadísticas descriptivas de las columnas numéricas de nuestro dataframe.



```
In [11]: # Completamos los datos nulos de la columna price con la media
         Ask.loc[:, 'price'].fillna(Ask['price'].mean(), inplace=True)

In [12]: Ask['price'] = Ask['price'].fillna(Ask['price'].mean())

In [14]: # Se depuran las columnas no requeridas
         columns_to_drop = ['id', 'timestamp', 'datetime', 'timezone', 'product_id', 'latitude', 'longitude']
         Ask.drop(columns=columns_to_drop, inplace=True)

In [15]: Ask.describe()
```

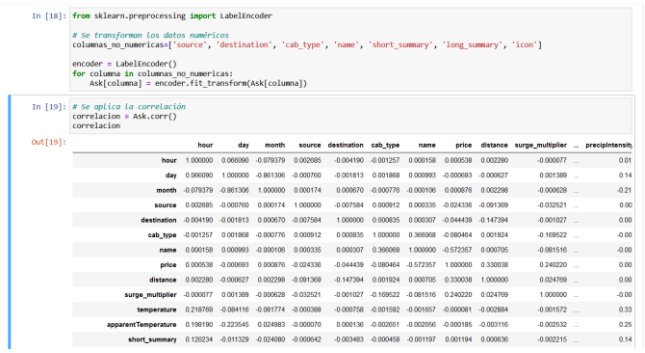
Out[15]:

	hour	day	month	price	distance	surge_multiplier	temperature	apparentTemperature	precipIntensity	precipPro
count	693071.000000	693071.000000	693071.000000	693071.000000	693071.000000	693071.000000	693071.000000	693071.000000	693071.000000	693071
mean	11.619137	17.794365	11.586684	16.545125	2.189430	1.013870	39.584388	35.881222	0.008922	0
std	6.948114	9.982286	0.492429	8.946089	1.138937	0.091641	6.728084	7.918707	0.026901	0
min	0.000000	1.000000	11.000000	2.500000	0.020000	1.000000	18.910000	12.130000	0.000000	0
25%	6.000000	13.000000	11.000000	9.500000	1.280000	1.000000	36.450000	31.910000	0.000000	0
50%	12.000000	17.000000	12.000000	16.000000	2.160000	1.000000	40.490000	35.900000	0.000000	0
75%	18.000000	28.000000	12.000000	22.500000	2.920000	1.000000	43.580000	40.080000	0.000000	0
max	23.000000	30.000000	12.000000	97.500000	7.860000	3.000000	57.220000	57.220000	0.144700	1

8 rows × 43 columns

Fig. 6. *Depuración de columnas*

El siguiente paso fue codificar nuestra variables categóricas a datos numéricos. Así que primero creamos una lista con aquellas columnas que queremos codificar y por medio de una iteración y de las funciones proporcionadas por sklearn, las transformamos a valores numéricos. Después de codificar las variables categóricas, se calcula la matriz de correlación entre todas las variables numéricas en el dataframe **Ask** utilizando el método **corr()** de pandas. La matriz de correlación muestra la relación lineal entre pares de variables, donde los valores cercanos a 1 indican una correlación positiva fuerte, los valores cercanos a -1 indican una correlación negativa fuerte, y los valores cercanos a 0 indican una correlación débil o nula.



```
In [18]: from sklearn.preprocessing import LabelEncoder

         # Se transforman los datos numéricos
         columnas_no_numericas=['source', 'destination', 'cab_type', 'name', 'short_summary', 'long_summary', 'icon']

         encoder = LabelEncoder()
         for columna in columnas_no_numericas:
             Ask[columna] = encoder.fit_transform(Ask[columna])

In [19]: # Se aplica la correlación
         correlacion = Ask.corr()
         correlacion
```

Out[19]:

	hour	day	month	source	destination	cab_type	name	price	distance	surge_multiplier	...	precipIntensity
hour	1.000000	0.066090	-0.079379	0.002685	-0.004190	-0.001257	0.000158	0.000538	0.002280	-0.000077	...	0.01
day	0.066090	1.000000	-0.861306	-0.000760	-0.001813	0.001868	0.000993	-0.000693	-0.000627	0.001389	...	0.14
month	-0.079379	-0.861306	1.000000	0.000174	0.000670	-0.000776	-0.000106	0.000876	0.002298	-0.000628	...	-0.21
source	0.002685	-0.000760	0.000174	1.000000	-0.007584	0.000912	0.000335	-0.024336	-0.091369	-0.032521	...	0.00
destination	-0.004190	-0.001813	0.000670	-0.007584	1.000000	0.000835	0.000307	-0.044439	-0.147394	-0.001027	...	0.00
cab_type	-0.001257	0.001868	-0.000776	0.000912	0.000835	1.000000	0.366068	-0.080464	0.001924	-0.169522	...	-0.00
name	0.000158	0.000993	-0.000106	0.000335	0.000307	0.366068	1.000000	-0.572357	0.000705	-0.081516	...	-0.00
price	0.000538	-0.000693	0.000876	-0.024336	-0.044439	-0.080464	-0.572357	1.000000	0.330038	0.240220	...	0.00
distance	0.002280	-0.000627	0.002298	-0.091369	-0.147394	0.001924	0.000705	0.330038	1.000000	0.024769	...	0.00
surge_multiplier	-0.000077	0.001389	-0.000628	-0.032521	-0.001027	-0.169522	-0.081516	0.240220	0.024769	1.000000	...	-0.00
temperature	0.218769	-0.084116	-0.081774	-0.000388	-0.000758	-0.001592	-0.001657	-0.000081	-0.002884	-0.001572	...	0.33
apparentTemperature	0.198190	-0.223545	0.024983	-0.000070	0.000136	-0.002651	-0.002056	-0.000185	-0.003116	-0.002532	...	0.25
short_summary	0.120234	-0.011329	-0.024080	-0.000642	-0.003483	-0.000458	-0.001197	0.001194	0.000636	-0.002215	...	0.14

Fig. 7*. Codificación de variables y su correlación*

Caja 3. Visualización de datos

Ahora en esta caja el código mostrará las visualizaciones que requerimos para nuestro análisis. Comenzaremos por importar la librería seaborn, la cual nos proporciona visualizaciones de datos, y después escribimos el código para generar el mapa de calor. Este mapa de calor nos proporciona información sobre la relación lineal entre todas las variables numéricas en el conjunto de datos. Esto nos proporciona una manera de identificar la correlación entre las variables de una manera visual, en donde las correlaciones más fuertes, ya sean positivas o negativas, se muestran en tonos más oscuros, mientras que las correlaciones más débiles se muestran en tonos más claros.

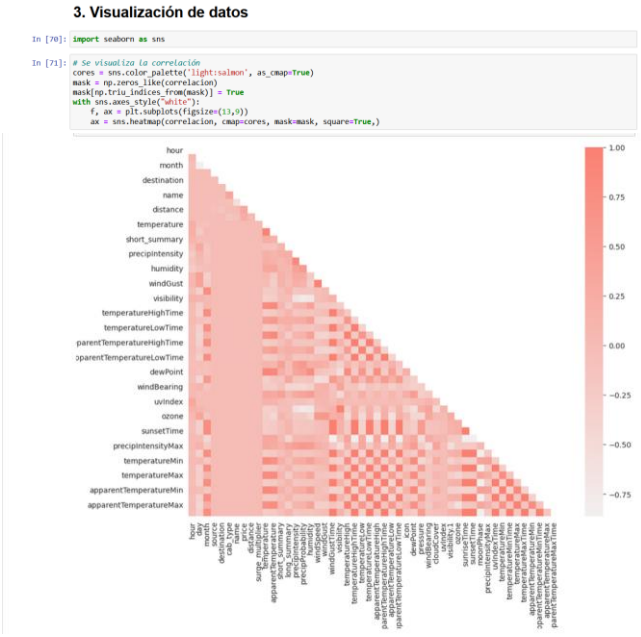
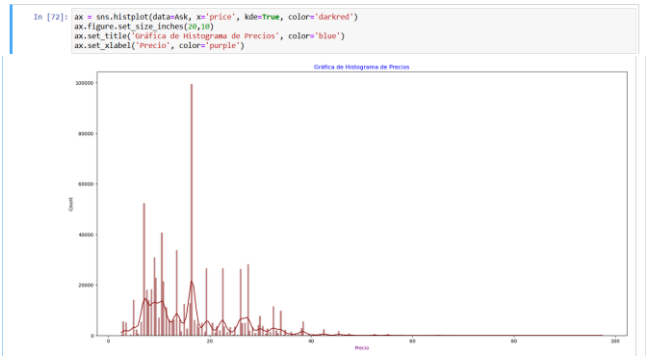


Fig. 8 *Mapa de calor de las variables*

Ahora haremos un histograma de precios, para ello volvemos a usar seaborn para la creación y personalización del histograma. El histograma nos permitirá visualizar la distribución y la tendencia central de los precios en el conjunto de datos.



```
In [72]: ax = sns.histplot(data=Ask, x='price', kde=True, color='darkred')
         ax.figure.set_size_inches(20,10)
         ax.set_title('Gráfica de Histograma de Precios', color='blue')
         ax.set_xlabel('Precio', color='purple')
```


Fig. 9. *Histograma de precios*

A continuación crearemos dos gráficos de dispersión en donde visualizamos la relación entre el tipo de taxi vs. su distancia y su precio. Aquí podremos observar la relación que tienen Uber y Lyft contra el precio y la distancia y podremos comparar, sacando las conclusiones que necesitemos. Para ello definimos una función llamada **grafica** para crear los gráficos de dispersión y para llamar los métodos necesarios para su creación, además de pasar los parámetros necesarios para apoyar la visualización.

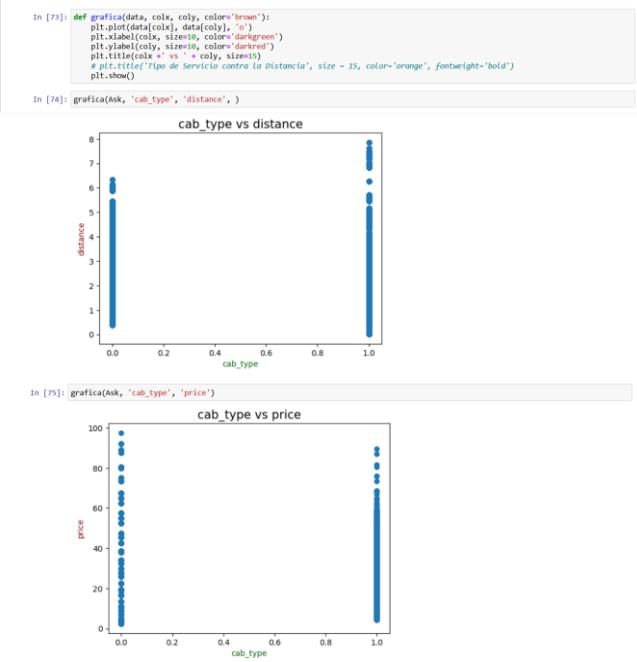


Fig. 10. *Gráficos de dispersión para el tipo de taxi vs la distancia y el precio (0 = Lyft, 1 = Uber)*

En seguida volveremos a crear un histograma esta vez para observar la distribución y la tendencia central de la variable distancia. Igualmente nos apoyaremos en seaborn para la creación y personalización del histograma. También nos permitirá identificar patrones o tendencias en los valores de distancia.

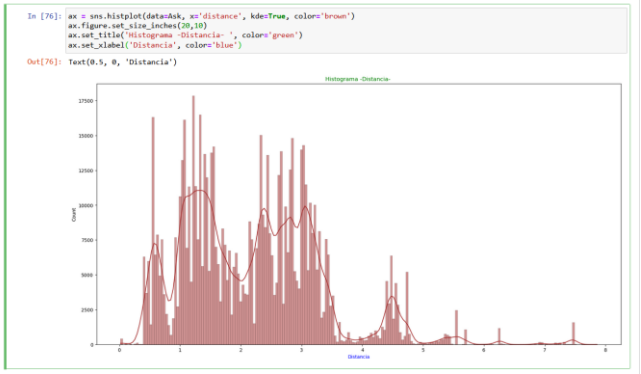


Fig. 11*. Histograma de distancia*

Nuestra última visualización será aplicar una curva logística a un gráfico de dispersión entre la variable cab_type y distancia, para observar cómo se comporta la relación entre las variables 'cab_type' y 'distancia' bajo el modelo de regresión logística. Para ello creamos las variables que harán de puntos de la recta y después graficamos. La línea sigmoide resultante, al tener ángulo de 0°, concluimos que no hay efecto de 'cab_type' en la variable de respuesta, o en otras palabras, 'cab_type' no tiene poder predictivo sobre la variable de interés, en este caso en la variable distancia.

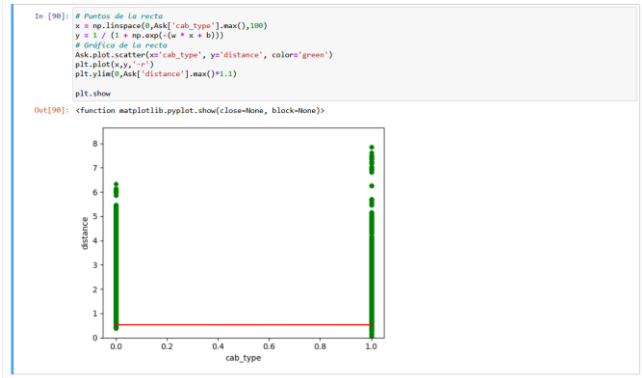


Fig. 12. *Gráfico de dispersión cab_type vs distance con curva logística*

Caja 4. Creación de matriz de atributos X y el vector de etiquetas y

A partir de esta caja estaremos preparando los datos para un modelo de aprendizaje, en este caso para la regresión logística. Para ello creamos la variable de destino o respuesta, y, para entrenar el modelo y para que represente lo que estamos tratando de predecir. Después creamos el dataframe X que contendrá las variables predictoras. Después por medio del método head() imprimimos una previa vista de las primeras filas de las variables que acabamos de crear.



Fig. 13. *Creación de variables X e y para nuestro modelo*

Caja 5.1. Escalado de los datos usando StandardScaler

Una vez creadas nuestras variables a usar para el modelo, nos enfocaremos en esta caja en el escalado de datos. El escalado de datos es una parte importante del preprocesamiento de datos en el aprendizaje automático ya que ayuda a mejorar la eficacia y el rendimiento de los modelos, además de otras razones cómo por ejemplo, ayudar a que converjan más rápido los algoritmos, evitar que las características con escalas más grandes dominen las que tienen escalas más pequeñas, comparar y analizar las importancias de las características de manera más significativa, entre otras razones.

En este caso se estarán usando dos técnicas de escalamiento: StandardScaler y MinMaxScaler. StandardScaler estandariza las características eliminando la media y escalando a la varianza unitaria. Esto significa que transforma las características para que tengan una media de 0 y una desviación estándar de 1. Así

creamos el objeto que realizará el escalado de datos. Después se utiliza el método **fit_transform()** para ajustar el escalador a los datos y transformar las variables predictoras (**X**) al mismo tiempo. Esto devuelve un arreglo que contiene las variables predictoras escaladas. Ahora se crea un dataframe utilizando los datos escalados y utilizando los mismos nombres de columnas. La última línea de código nos mostrará un resumen estadístico del dataframe creado.



Fig. 14. *Escalado de datos con StandardScaler*

Caja 5.1. Escalado de los datos usando MinMaxScaler

Aquí estaremos realizando básicamente lo mismo, sólo que ahora usaremos la técnica de MinMaxScaler para el escalado de datos. MinMaxScaler transforma las características escalándolas a un rango específico, generalmente entre 0 y 1. Esto se logra restando el valor mínimo y luego dividiendo por el rango (es decir, la diferencia entre el máximo y el mínimo). Entonces volvemos a crear el objeto para usar el escalado, utilizamos el método **fit_transform()** para transformar las variables, creamos el dataframe con los datos escalados y generamos una tabla con las primeras filas del dataframe creado y un resumen estadístico.



Fig. 15*. Escalado de datos con MinMaxScaler*

Caja 6. Entrenamiento y evaluación del modelo de regresión logística

En esta porción de código estamos construyendo una función llamada **regression** que lleva a cabo el entrenamiento y evaluación del modelo de regresión logística utilizando las características escaladas (X_scaled) y la variable de destino (y). A continuación se describe lo que se está haciendo. Primero se escalan las características utilizando StandardScaler. Luego dividimos los datos escalados en conjuntos de entrenamiento y prueba, esto nos permitirá evaluar el rendimiento del modelo en datos no vistos. A continuación creamos el objeto de regresión logística y se ajusta a los datos de entrenamiento. Se realizan las predicciones utilizando el modelo entrenado en los datos de prueba.

Ahora se calculan las métricas de evaluación del modelo, estas son: precisión, matriz de confusión, sensibilidad y puntuación F1. Por último, las imprimimos. Estas nos ayudarán a conocer el rendimiento de nuestro modelo y su capacidad para hacer predicciones precisas.

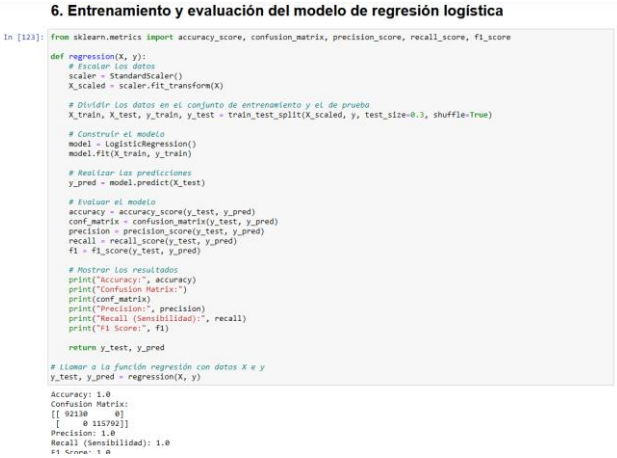


Fig. 16*. Usando nuestro modelo de regresión logística y calculando las métricas para conocer su rendimiento*

Caja 7.1. Probar la regresión con los datos estandarizados (StandardScaler)

En esta pequeña porción de código solo estamos probando nuestro modelo haciendo uso los datos estandarizados por la técnica StandardScaler. Así que solo llamaos a la función **regression** y haciendo uso de las variables de destino y las escaladas para obtener las métricas.

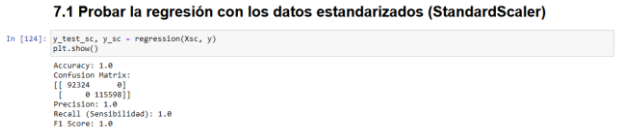


Fig. 17. *Regresión con datos escalados (StandardScaler)*

Caja 7.2. Probar la regresión con los datos estandarizados (MinMaxScaler)

Llevamos a cabo el mismo procedimiento pero ahora usando la técnica de MinMaxScaler.

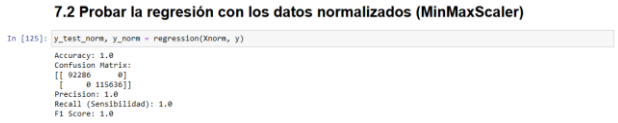


Fig. 18. Regresión con datos escalados (MinMaxScaler)

CONCLUSIONES

Podemos observar como haciendo uso de las librerías y las funciones y métodos de Python podemos realizar análisis detallados de cualquier conjunto de datos que queramos. También se pudo observar todo el proceso que se requiere para poder evaluar y usar un modelo de machine learning, en este caso una regresión logística. Desde la carga de los datos, su preprocesamiento, su escalado, el entrenamiento, el uso del modelo para las predicciones y el cálculo de las métricas para evaluar el rendimiento de nuestro modelo.

A continuación se presentan los resultados:

Accuracy (Precisión): Significa que el porcentaje de predicciones correctas de todas las predicciones realizadas es del 100%. Todas las muestras en el conjunto de prueba se clasificaron correctamente.

Confusion Matrix: Muestra que no hubo falsos positivos en el conjunto de prueba. Todos los datos negativos fueron correctamente identificados como negativos, y todos los datos positivos fueron correctamente identificados como positivos.

Recall: Significa que el porcentaje de muestras positivas que fueron correctamente identificadas por el modelo es del 100%. No hubo falsos negativos en las predicciones

F1 Score: El puntaje de F1 indica que no hubo errores ni falsos positivos ni falsos negativos en las predicciones

REFERENCIAS

Rajesh, R. (10 de septiembre de 2020). *Data Science: Data Cleansing and Visualization for Beginners Using Python.* Medium. https://rajesh-r.medium.com/data-science-data-cleansing-and-visualization-for-beginners-using-python-3f55323768f1

Li, Susan (28 de septiembre de 2017). *Building A Logistic Regression in Python, Step by Step*. Medium. https://towardsdatascience.com/building-a-logistic-regression-in-python-step-by-step-becd4d56c9c8

Grus, Joel (2019). *Data Science from Scratch. First Principles with Python.* O'Reilly